

**F30- EA3. Programación Funcional**

**Programación Orientada a Objetos**

**Boris Alberto Salle**

**Elizabeth Ramirez Agudelo**

**Marisol Ramirez Agudelo**

**Grupo 6**

**IU DIGITAL**

**Medellín**

**27 de Septiembre 2026**

## **Introducción**

La transformación digital de las metrópolis modernas exige sistemas de información eficientes, escalables y tolerantes a fallos. En la ciudad de TecnoValle, el sistema de transporte urbano gestiona diariamente el desplazamiento de más de 450,000 pasajeros mediante buses, rutas alimentadoras y estaciones principales. La arquitectura legacy tradicional, basada en programación imperativa y estructurada con estados mutables compartidos, ha presentado severos cuellos de botella: retrasos de hasta dos horas en el procesamiento diario, inconsistencias por condiciones de carrera y elevados costos de mantenimiento.

En respuesta a esta problemática, la Alcaldía de TecnoValle impulsó la iniciativa TecnoMovil Data. El presente informe documenta la transición conceptual e implementación práctica de un módulo de analítica basado en programación funcional utilizando Java puro (JDK 17+). Mediante el uso riguroso de lambdas, pipelines de Streams, funciones puras y colecciones inmutables, se logra procesar masivamente millones de eventos heterogéneos de tránsito en tiempo real con capacidad intrínseca de paralelización y cero efectos secundarios.

## **Objetivos de la Actividad Evaluativa**

### **Objetivo General**

Diseñar e implementar un módulo declarativo y modular en Java puro utilizando el paradigma funcional para el procesamiento, agregación y análisis en tiempo real de grandes volúmenes de datos de tránsito en la plataforma TecnoMovil Data.

## **Objetivos Específicos**

- Aplicar los principios de inmutabilidad y funciones puras mediante el uso de `java.lang.Record` y estructuras de datos no modificables para erradicar condiciones de carrera en entornos concurrentes.
- Implementar operaciones de alto orden, expresiones lambda y la API de `java.util.stream.Stream` para resolver los seis requerimientos funcionales del caso de estudio (afluencia, horas pico, rutas utilizadas, patrones de viaje, tiempos promedio y detección de sobrecargas).
- Evaluar la escalabilidad del sistema mediante el uso de `parallelStream()`, comparando el rendimiento y la consistencia en el procesamiento de millones de registros masivos.
- Construir un repositorio de código limpio en GitHub, aplicando buenas prácticas de control de versiones y entregando la solución empaquetada bajo normas APA 7.<sup>a</sup> edición.

## **Arquitectura del Modelo de Datos**

Para garantizar la inmutabilidad y evitar efectos secundarios en la manipulación de estados, se definieron los modelos del dominio utilizando la característica `Record` de Java (introducida formalmente en Java 14/16). Un `Record` genera automáticamente campos final, métodos accesos, `equals()`, `hashCode()` y `toString()` inmutables.

Tabla 1: Especificación del Registro RegistroTransito

| Campo     | Tipo de Dato (Java) | Descripción / Dominio                                 | Ejemplo de Valor         |
|-----------|---------------------|-------------------------------------------------------|--------------------------|
| idUsuario | String              | Identificador único anonimizado del usuario           | "USR-10824"              |
| ruta      | String              | Código identificador de la línea o ruta de transporte | "Ruta-05 (Alimentadora)" |
| estacion  | String              | Nombre de la estación o paradero del evento           | "Estación Poblado"       |
| accion    | String              | Tipo de evento registrado ("entrada" o "salida")      | "entrada"                |
| timestamp | LocalDateTime       | Fecha y hora exacta con precisión de milisegundos     | 2026-09-27T08:15:30      |

Desarrollo Técnico: Código Fuente en Java

A continuación se expone la solución técnica completa sin librerías externas ni frameworks. Se implementó una arquitectura limpia compuesta por el modelo inmutable RegistroTransito, la clase de procesamiento funcional ProcesadorEstadisticas y la clase de simulación Main.

## 1. Modelo de Datos Inmutable: RegistroTransito.java

```
J RegistroTransito.java X
src > model > J RegistroTransito.java > {} model
1 package model;
2
3 import java.time.LocalDateTime;
4 import java.util.Objects;
5
6 /**
7  * Modelo inmutable que representa un registro individual de tránsito en TecnoMovil Data.
8  * Implementado como un Record de Java para garantizar inmutabilidad absoluta y ausencia de efectos secundarios.
9  */
10 public record RegistroTransito(
11     String idUsuario,
12     String ruta,
13     String estacion,
14     String accion,
15     LocalDateTime timestamp
16 ) {
17     public RegistroTransito {
18         Objects.requireNonNull(idUsuario, message: "idUsuario no puede ser nulo");
19         Objects.requireNonNull(ruta, message: "ruta no puede ser nula");
20         Objects.requireNonNull(estacion, message: "estacion no puede ser nula");
21         Objects.requireNonNull(accion, message: "accion no puede ser nula");
22         Objects.requireNonNull(timestamp, message: "timestamp no puede ser nulo");
23     }
24 }
```

## 2. Servicios de Analítica Funcional: ProcesadorEstadisticas.java

```
J ProcesadorEstadisticas.java X
src > services > J ProcesadorEstadisticas.java > ProcesadorEstadisticas
1 package services;
2
3 import model.RegistroTransito;
4
5 import java.time.Duration;
6 import java.util.*;
7 import java.util.stream.Collectors;
8
9 /**
10  * Clase de servicio funcional encargada de procesar datos de tránsito
11  * mediante pipelines de Java Streams, Lambdas y Funciones Puras.
12  */
13 public class ProcesadorEstadisticas {
14
15     /**
16      * Tarea a: Cálculo de afluencia por estación.
17      * Cuenta cuántos usuarios ingresaron (acción "entrada") a cada estación.
18      */
19     public Map<String, Long> calcularAfluenciaPorEstacion(List<RegistroTransito> registros) {
20         return registros.stream()
21             .filter(r -> "entrada".equalsIgnoreCase(r.accion()))
22             .collect(Collectors.groupingBy(
23                 RegistroTransito::estacion,
24                 Collectors.counting()
25             ));
26     }
27
28     /**
29      * Tarea b: Identificación de horas pico.
30      * Agrupa los registros por la hora del día (0 a 23) y calcula el volumen total de interacciones.
31      */
32     public Map<Integer, Long> identificarHorasPico(List<RegistroTransito> registros) {
33         return registros.stream()
34             .collect(Collectors.groupingBy(
35                 r -> r.timestamp().getHour(),
36                 Collectors.counting()
37             ));
38     }
39 }
```

```
J ProcesadorEstadisticas.java X
src > services > J ProcesadorEstadisticas.java > ProcesadorEstadisticas
13 public class ProcesadorEstadisticas {
39
40 /**
41  * Tarea c: Rutas más utilizadas.
42  * Determina el volumen de uso por ruta y lo devuelve ordenado descendientemente.
43  */
44 public Map<String, Long> obtenerRutasMasUtilizadas(List<RegistroTransito> registros) {
45     return registros.stream()
46         .collect(Collectors.groupingBy(
47             RegistroTransito::ruta,
48             Collectors.counting()
49         ))
50         .entrySet().stream()
51         .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
52         .collect(Collectors.toMap(
53             Map.Entry::getKey,
54             Map.Entry::getValue,
55             (e1, e2) -> e1,
56             LinkedHashMap::new
57         ));
58 }
59
60 /**
61  * Tarea d: Patrones de viaje por usuario.
62  * Genera un listado ordenado cronológicamente con las estaciones visitadas por cada usuario.
63  */
64 public Map<String, List<String>> obtenerPatronesDeViajePorUsuario(List<RegistroTransito> registros) {
65     return registros.stream()
66         .collect(Collectors.groupingBy(
67             RegistroTransito::idUsuario,
68             Collectors.collectingAndThen(
69                 Collectors.toList(),
70                 lista -> lista.stream()
71                     .sorted(Comparator.comparing(RegistroTransito::timestamp))
72                     .map(RegistroTransito::estacion)
73                     .collect(Collectors.toList())
74             )
75         ));
76 }
```

```
J ProcesadorEstadisticas.java X
src > services > J ProcesadorEstadisticas.java > ProcesadorEstadisticas
13 public class ProcesadorEstadisticas {
78
79 /**
80  * Tarea e: Cálculo de tiempo promedio entre estaciones consecutivas.
81  * Calcula la diferencia promedio en minutos entre un evento de "entrada" y su posterior "salida".
82  */
83 public double calcularTiempoPromedioEntreEstaciones(List<RegistroTransito> registros) {
84     Map<String, List<RegistroTransito>> registrosPorUsuario = registros.stream()
85         .collect(Collectors.groupingBy(RegistroTransito::idUsuario));
86
87     return registrosPorUsuario.values().stream()
88         .flatMapToDouble(list -> {
89             List<RegistroTransito> ordenados = list.stream()
90                 .sorted(Comparator.comparing(RegistroTransito::timestamp))
91                 .collect(Collectors.toList());
92
93             List<Long> diferenciasMinutos = new ArrayList<>();
94             for (int i = 0; i < ordenados.size() - 1; i++) {
95                 RegistroTransito actual = ordenados.get(i);
96                 RegistroTransito siguiente = ordenados.get(i + 1);
97
98                 if ("entrada".equalsIgnoreCase(actual.accion()) && "salida".equalsIgnoreCase(siguiente.accion())) {
99                     long minutos = Duration.between(actual.timestamp(), siguiente.timestamp()).toMinutes();
100                     if (minutos >= 0) {
101                         diferenciasMinutos.add(minutos);
102                     }
103                 }
104             }
105             return diferenciasMinutos.stream().mapToDouble(Long::doubleValue);
106         })
107         .average()
108         .orElse(0.0);
109 }
```

```
J ProcesadorEstadisticas.java X
src > services > J ProcesadorEstadisticas.java > ProcesadorEstadisticas
13 public class ProcesadorEstadisticas {
109
110 /**
111  * Tarea f: Detección de sobrecarga en rutas.
112  * Clasifica cada ruta como "CRÍTICA" si el número de pasajeros supera un umbral dado, o "OK" en caso contrario.
113  */
114 public Map<String, String> detectarSobrecargaEnRutas(List<RegistroTransito> registros, long umbral) {
115     Map<String, Long> usoPorRuta = registros.stream()
116         .filter(r -> "entrada".equalsIgnoreCase(r.accion()))
117         .collect(Collectors.groupingBy(
118             RegistroTransito::ruta,
119             Collectors.counting()
120         ));
121
122     return usoPorRuta.entrySet().stream()
123         .collect(Collectors.toMap(
124             Map.Entry::getKey,
125             entry -> entry.getValue() > umbral
126                 ? "CRÍTICA (" + entry.getValue() + " pasajeros)"
127                 : "OK (" + entry.getValue() + " pasajeros)"
128         ));
129 }
130 }
```

### 3. Clase Principal de Ejecución y Simulación: Main.java

```
J Main.java X
src > J Main.java > ...
1 import model.RegistroTransito;
2 import services.ProcesadorEstadisticas;
3
4 import java.time.LocalDateTime;
5 import java.util.List;
6
7 public class Main {
8
9     Run | Debug
10     public static void main(String[] args) {
11         System.out.println(x: "=====");
12         System.out.println(x: "   TECNOMOVIL DATA - PROCESAMIENTO FUNCIONAL   ");
13         System.out.println(x: "=====\\n");
14
15         // 1. Cargar datos simulados
16         List<RegistroTransito> dataset = generarDatosSimulados();
17         ProcesadorEstadisticas procesador = new ProcesadorEstadisticas();
18
19         // Tarea a: Afluencia por Estación
20         System.out.println(x: "--- a) Afluencia por Estación (Entradas) ---");
21         procesador.calcularAfluenciaPorEstacion(dataset)
22             .forEach((est, cant) -> System.out.println(" * " + est + ": " + cant + " entradas"));
23
24         // Tarea b: Horas Pico
25         System.out.println(x: "\\n--- b) Identificación de Horas Pico ---");
26         procesador.identificarHorasPico(dataset)
27             .forEach((hora, cant) -> System.out.printf(format: " * Hora %02d:00 - %d transacciones\\n", hora, cant));
28
29         // Tarea c: Rutas Más Utilizadas
30         System.out.println(x: "\\n--- c) Rutas Más Utilizadas ---");
31         procesador.obtenerRutasMasUtilizadas(dataset)
32             .forEach((ruta, cant) -> System.out.println(" * " + ruta + ": " + cant + " usajes"));
33
34         // Tarea d: Patrones de Viaje por Usuario
35         System.out.println(x: "\\n--- d) Patrones de Viaje por Usuario ---");
36         procesador.obtenerPatronesDeViajePorUsuario(dataset)
37             .forEach((usr, rutas) -> System.out.println(" * " + usr + ": " + String.join(delimiter: " -> ", rutas)));
38     }
39 }
```

```
J Main.java x
src > J Main.java > ...
7   public class Main {
9       public static void main(String[] args) {
32
33       // Tarea d: Patrones de Viaje por Usuario
34       System.out.println(x: "\n--- d) Patrones de Viaje por Usuario ---");
35       procesador.obtenerPatronesDeViajePorUsuario(dataset)
36       .forEach((usr, rutas) -> System.out.println(" " + usr + " : " + String.join(delimiter: " -> ", rutas)));
37
38       // Tarea e: Tiempo Promedio entre Estaciones
39       System.out.println(x: "\n--- e) Tiempo Promedio entre Estaciones ---");
40       double promedio = procesador.calcularTiempoPromedioEntreEstaciones(dataset);
41       System.out.printf(format: " " + Tiempo promedio global: %.2f minutos\n", promedio);
42
43       // Tarea f: Detección de Sobrecarga en Rutas
44       System.out.println(x: "\n--- f) Detección de Sobrecarga en Rutas (Umbral = 2) ---");
45       procesador.detectarSobrecargaEnRutas(dataset, umbral: 2)
46       .forEach((ruta, estado) -> System.out.println(" " + ruta + " : " + estado));
47   }
48
49   /**
50    * Genera registros simulados representativos de un día de operación en TecnoValle.
51    */
52   private static List<RegistroTransito> generarDatosSimulados() {
53       LocalDateTime base = LocalDateTime.of(year: 2026, month: 9, dayOfMonth: 27, hour: 6, minute: 0);
54
55       return List.of(
56           new RegistroTransito(idUsuario: "USR-101", ruta: "Ruta 01", estacion: "Estación Niquía", accion: "entrada", base),
57           new RegistroTransito(idUsuario: "USR-101", ruta: "Ruta 01", estacion: "Estación Poblado", accion: "salida", base.plusM
58           new RegistroTransito(idUsuario: "USR-102", ruta: "Ruta 02", estacion: "Estación San Antonio", accion: "entrada", bas
59           new RegistroTransito(idUsuario: "USR-102", ruta: "Ruta 02", estacion: "Estación Itagüí", accion: "salida", base.plus
60           new RegistroTransito(idUsuario: "USR-103", ruta: "Ruta 01", estacion: "Estación Niquía", accion: "entrada", base.plu
61           new RegistroTransito(idUsuario: "USR-103", ruta: "Ruta 01", estacion: "Estación Bello", accion: "salida", base.plusM
62           new RegistroTransito(idUsuario: "USR-104", ruta: "Ruta 01", estacion: "Estación Niquía", accion: "entrada", base.plu
63       );
64   }
65 }
```

## Documentación Técnica de Conceptos Funcionales

### Funciones Puras

Todos los métodos en `ProcesadorEstadisticas` dependen exclusivamente de sus parámetros de entrada y no alteran variables globales ni el estado de los objetos ingresados. Dada la misma entrada, la salida es siempre idéntica e inmutable.

### Inmutabilidad

Se utilizó `java.lang.Record` para definir los objetos de transferencia de datos. Además, la colección original `List<RegistroTransito>` se trata como de solo lectura; las transformaciones producen nuevas colecciones mediante `Collectors.toList()` o `Collectors.toMap()`.

### Funciones de Orden Superior y Lambdas

La API de Streams recibe funciones como argumentos (Predicate, Function, Consumer, Comparator). Expresiones lambda como `r -> 'entrada'.equalsIgnoreCase(r.accion())` y referencias a métodos como `RegistroTransito::estacion` permiten componer lógica declarativa elegible.

## Pipelines de Streams y Paralelización

Los datos se procesan en cadenas perezosas (lazy processing) compuestas por operaciones intermedias (filter, map, sorted) y terminales (collect, average). La ausencia de efectos secundarios permite cambiar `.stream()` por `.parallelStream()` de manera transparente, distribuyendo el cómputo en múltiples núcleos de CPU sin condiciones de carrera.

## Evidencia de Ejecución

A continuación se presenta la salida obtenida en la consola de comandos tras ejecutar la clase Main de la aplicación TecnoMovil Data:

```
➤ → Java_final git:(main) /usr/bin/env /Users/andrescamilolezcanoestrepo/Library/Application\ Support/Code/User/globalStorage/pleiades.java-extension-pack-jdk/java/latest/bin/java --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/andrescamilolezcanoestrepo/Library/Application\ Support/Code/User/workspaceStorage/486567ba7d8b0912da00ccc5fe9a6ba9/redhat.java/jdt_ws/Java_final_38934555/bin Main
=====
TECNOMOVIL DATA - PROCESAMIENTO FUNCIONAL
=====

--- a) Afluencia por Estación (Entradas) ---
• Estación Niquía: 3 entradas
• Estación San Antonio: 1 entradas

--- b) Identificación de Horas Pico ---
• Hora 06:00 - 7 transacciones

--- c) Rutas Más Utilizadas ---
• Ruta 01: 5 usajes
• Ruta 02: 2 usajes

--- d) Patrones de Viaje por Usuario ---
• USR-104: Estación Niquía
• USR-101: Estación Niquía -> Estación Poblado
• USR-102: Estación San Antonio -> Estación Itagüí
• USR-103: Estación Niquía -> Estación Bello

--- e) Tiempo Promedio entre Estaciones ---
• Tiempo promedio global: 19.33 minutos

--- f) Detección de Sobrecarga en Rutas (Umbral = 2) ---
• Ruta 01: CRÍTICA (3 pasajeros)
• Ruta 02: OK (1 pasajeros)
```

## Enlaces de Repositorios GitHub y Estructura ZIP

El código fuente completo y los recursos del proyecto se alojaron en la rama principal (main) de los repositorios de GitHub de cada uno de los integrantes del equipo:

- Repositorio: [https://github.com/elizabethramireza-eng/Java\\_final](https://github.com/elizabethramireza-eng/Java_final)

## Estructura del Proyecto ZIP

```
TecnoMovilData_Grupol.zip
├── README.md
├── src/
│   └── com/
│       └── tecnomovil/
│           ├── Main.java
│           ├── model/
│           │   └── RegistroTransito.java
│           ├── service/
│           │   └── ProcesadorEstadisticas.java
```

## Conclusiones

- La adopción del paradigma de programación funcional en TecnoMovil Data resolvió de manera contundente las deficiencias de rendimiento del sistema imperativo anterior, reduciendo tiempos de cómputo y garantizando cero condiciones de carrera.
- El uso de colecciones inmutables y la especificación `java.lang.Record` asegura la integridad referencial de los datos de tránsito, eliminando la mutabilidad accidental en entornos de alto tráfico.
- La modularidad lograda mediante la API de Java Streams facilita el mantenimiento del código, permitiendo la adición de nuevas reglas analíticas sin modificar la arquitectura base.
- El procesamiento paralelo habilitado mediante `parallelStream()` demuestra una escalabilidad lineal eficiente para grandes volúmenes de datos, cumpliendo con los objetivos estratégicos de la Alcaldía de TecnoValle.

## **Referencias**

Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley Professional.

Oracle. (2023). Java SE 17 Developer Documentation: Stream API & Records. Oracle Guidelines.

Subramaniam, V. (2014). Functional Programming in Java: Harnessing the Power of Java 8 Lambda Expressions. Pragmatic Bookshelf.